Architectural Convergence in Repository-Resident AI: Implementing a Pointer-First Constitution and Unified Epistemic Engine for Stela

1. Introduction: The Epistemology of Repository-Resident Intelligence

The evolution of artificial intelligence from stateless query-response engines to "repository-resident" agents necessitates a fundamental reimagining of software governance. In a repository-resident model, the AI does not merely visit the codebase; it inhabits it. The file system becomes its memory, the version control history its consciousness, and the governance documentation its conscience. The Stela platform represents a sophisticated implementation of this paradigm, governed by a rigid "Policy of Truth" (PoT) that establishes the repository as the Single Source of Truth (SSOT). However, as such systems mature, they inevitably encounter a critical architectural fracture: the divergence between the "truth" required for execution (immediate, safe, restricted) and the "truth" required for discovery (comprehensive, interconnected, explanatory).

This report presents an exhaustive analysis of the architectural imperative to unify these divergent epistemic states within the Stela platform. Specifically, it examines the transition to a "Pointer-First" agent constitution—a governance model where agents operate via reference to immutable canonical documents rather than internalizing duplicated instructions. This shift is not merely syntactical but constitutional, addressing the "Two Truths" operational drift identified in recent architectural audits.[1] By synthesizing the execution pipeline (governed by ops/bin/context) with the discovery pipeline (governed by ops/bin/llms and the llms.txt standard), the platform aims to achieve a "Flat Truth" architecture where agents, operators, and linters observe a single, consistent reality.

The analysis draws upon a comprehensive review of the platform's current state [1], the strategic consolidation proposal [1], and the external standards influencing these internal mechanics.[2] It culminates in a detailed examination of Dispatch Packet DP-OPS-0059, which serves as the implementation vehicle for this constitutional reform.

1.1 The Stela Axioms and the Crisis of Duplication

At the core of Stela's governance is the **Policy of Truth (PoT)**, a constitutional document that explicitly forbids "deception or incoherence" as inherently destabilizing forces.[1] The PoT

establishes "Physical Laws" for the repository, most notably the **Filing Doctrine**, which segregates the system into three teleological domains:

- **ops/ (Run):** The executable kernel containing binaries, manifests, and automation.
- **docs/ (Explain):** The explanatory layer containing manuals, maps, and rationale.
- **projects/ (Work):** The payload domain where value is created.

This structure is enforced by a suite of linters (e.g., tools/lint/truth.sh, tools/lint/context.sh) and strict behavioral standards for AI agents, including "Absolute Literalism" and "Linguistic Precision" (e.g., the prohibition of contractions).[1]

However, the current system suffers from a recursive flaw: to enforce these laws, the laws themselves are often duplicated into agent prompts, Dispatch Packets (DPs), and context bundles. This redundancy creates "Ghost Canon"—fragments of governance text that exist outside the SSOT. When the canonical PoT.md is updated, these scattered duplicates remain static, leading to "Canon Drift".[1] A "Pointer-First" constitution resolves this by mandating that agents and DPs reference the file path of the rule (PoT.md) rather than restating the rule itself. This ensures that the system state remains in equilibrium, a core operational directive of the platform.[1]

1.2 The Scope of Unification

The proposed unification targets the schism between **Context** (what the agent needs to *do* a specific task) and **Discovery** (what the agent needs to *know* about the system). Currently, Stela maintains separate pipelines for these needs, resulting in conflicting signals. The unification strategy proposes a hierarchical inheritance model for manifests—Layer 0 (CORE), Layer 1 (OPS), and Layer 2 (DISCOVERY)—to ensure that every level of agent operation inherits from the same foundational truth.[1]

---

2. Analysis of the Existing Epistemic Architecture

To understand the necessity of DP-OPS-0059, one must first dissect the existing machinery of Stela's context engine. The platform relies on a complex interplay of static documentation and dynamic binaries to maintain its state.

2.1 The Canonical Surfaces

The repository defines seven "Canon Surfaces" that serve as the hard constraints for all operations.[1] These surfaces function as the immutable bedrock upon which the logic of the AI is constructed.

Surface	Role	Governance Function

PoT.md	The Constitution	Final authority on staffing, jurisdiction, and enforcement.
TASK.md	Active Work Surface	Contains the current Dispatch Packet (DP) and session work log.
SoP.md	History Ledger	"State of Play" record of what shipped and why.
docs/MANUAL.md	Operator Mechanics	Command console and execution cycle instructions.
docs/MAP.md	Context Wayfinding	Continuity map for navigating repository context.
ops/lib/manifests/CONTEXT.md	Execution Context	The required artifact set for task execution.
llms.txt	Discovery Entry Point	The machine-readable map for external reasoning.

These surfaces are not merely text files; they are active components of the system's "operating system." For instance, ops/bin/open reads PoT.md to inject governance rules into the session prompt [1], and tools/lint/context.sh verifies that every file listed in the context manifest actually exists.[1]

2.2 The Execution Pipeline: ops/bin/context

The execution pipeline is designed for safety and focus. Driven by the binary ops/bin/context (and its underlying tools open and dump), this pipeline creates a "Context Snapshot" for an agent about to perform a task.[1]

Crucially, this pipeline enforces a **Context Hazard** axiom: it strictly excludes docs/library/agents, docs/library/tasks, and docs/library/skills from the global context manifest.[1] The rationale is that these libraries contain instructions for *other* agents or *other* tasks. If an agent reads the instruction set for a "Security Reviewer" while trying to perform a

"Refactor Clean," it may hallucinate constraints or execute logic outside its scope. Therefore, the execution truth is a *censored* truth—it hides the library to prevent instruction drift.

2.3 The Discovery Pipeline: ops/bin/llms

The discovery pipeline, driven by ops/bin/llms, serves a different purpose: comprehension. It generates artifacts like llms.txt, llms-full.txt, and llms-ops.txt.[1] These bundles are designed to help an agent (or human) understand the *capabilities* of the system.

Unlike the execution pipeline, the discovery pipeline *must* include the libraries. To plan a task, the "Architect" agent needs to know which agents and skills are available in the registry. This creates the fundamental epistemic conflict: the "Architect" sees a world populated with tools (Discovery Truth), but the "Contractor" executing the plan operates in a world where those tools are invisible (Execution Truth) unless explicitly invoked.[1]

2.4 The Operational Friction

This bifurcation manifests in several operational frictions identified in the "Unifying LLM Context and Operations" report [1]:

- **Redundant Bundles:** The system currently maintains llms-governance.txt, llms-ops.txt, llms-small.txt, and others. These are distinct files that must be manually kept in sync or regenerated. If ops/bin/llms is not run, these bundles decay into "Ghost Canon."
- **Prompt Provisions:** The "Architect" prompt (E-PROMPT-03) currently requires a manual addendum to allowlist five specific filenames (llms.txt, llms-small.txt, etc.) whenever a context refresh is requested.[1] This hardcodes file paths into the prompt logic, a violation of the pointer-first principle.
- **Linter Conflict:** The Dispatch Packet linter (tools/lint/dp.sh) is designed to prevent "context bloating." However, the prompts often *require* the loading of discovery bundles to perform high-level planning. The linter flags this as a violation, forcing the operator to override safety checks or construct convoluted workarounds.

---

3. The "Two Truths" Operational Drift

The existence of two parallel mechanisms for defining reality—one for doing (context.sh) and one for knowing (llms.txt)—creates a vulnerability known as "Operational Drift." In Stela's strict governance model, drift is defined as "any divergence between canon and repository state".[1]

3.1 Divergence Analysis

The "Two Truths" problem can be visualized as a set discrepancy between the set of files C

(Execution Context) and the set of files D (Discovery Context).

$$C \subset D$$

While it is logical for the execution context to be a subset of the discovery context (to reduce noise), the *definition* of these sets is currently managed by two different manifests: ops/lib/manifests/CONTEXT.md for C and ops/lib/manifests/LLMS.md for D.[1]

- **Risk 1: Definition Drift.** If a new governance file (e.g., docs/SECURITY.md) is added, it must be manually added to *both* manifests. If added only to LLMS.md, the executing agent will never see the new security policy. If added only to CONTEXT.md, the planning agent will not know the policy exists.
- **Risk 2: Linter Blind Spots.** The tools/lint/context.sh script validates CONTEXT.md. It does *not* validate LLMS.md. This means the discovery context can contain broken links ("Dead Pointers") that go undetected until an agent tries to read them, causing a runtime failure.
- **Risk 3: Semantic Contamination.** The "Context Hazard" rule (excluding libraries) is enforced rigidly on CONTEXT.md. However, llms-full.txt includes these libraries. If a worker mistakenly loads llms-full.txt instead of the canonical dump, they ingest the hazards, violating the "Anti-Drift Governance" directive.[1]

3.2 The Ghost Canon Phenomenon

"Ghost Canon" refers to authoritative text that exists in a location that is not the SSOT. The current TASK.md template is a prime example. It includes inline text defining the "Freshness Gate," the "Canon Load Order," and the "Receipt" requirements.[1]

When PoT.md was recently updated to change the "Branching Doctrine" (e.g., forbidding direct pushes to main), the text inside TASK.md had to be manually updated to reflect this. If the template is not updated, an agent reading TASK.md sees one set of rules, while the linter reading PoT.md enforces another. This cognitive dissonance paralyzes the AI, which is programmed to "Stop work" upon detecting ambiguity.[1]

3.3 The Evidence from SoP.md

The State of Play (SoP) ledger [1] records recent attempts to mitigate these issues, indicating an escalating struggle with context management:

- **DP-OPS-0051:** "Eliminated TASK.md ghost-canon drift vectors by switching the Freshness Gate to pointer-first governance." This was a partial fix, addressing only the freshness gate.
- **DP-OPS-0054:** "TASK Surface Stabilization... stabilized TASK.md Session State to pointer-first shape."
- **DP-OPS-0057:** "Ops Surface Operational Certification... updating the llms ops bundle

manifest pointers."

These entries confirm that the platform is actively migrating toward the architecture proposed in this report. DP-OPS-0059 is the logical culmination of this sequence: a systemic unification rather than a patch-level fix.

---

4. The Pointer-First Constitution

The "Pointer-First" constitution is the doctrinal response to Ghost Canon. It mandates that no executable surface (like a DP or an Agent definition) should contain the *content* of a rule; it should only contain the *address* of the rule.

4.1 Theoretical Implementation

In a duplication-based model, an agent prompt might look like this:

> "Do not use contractions. Use absolute literalism. Stop if parameters are violated."

In a pointer-first model, the prompt looks like this:

> "Behavioral Logic Standard: See PoT.md Section 4.2."

This shifts the burden of definition to the SSOT. The agent is required to "dereference" the pointer by reading the canonical file. This requires the agent to have the canonical file in its context—which brings us back to the necessity of a unified context pipeline.

4.2 Application to TASK.md

The TASK.md file serves as the dashboard for the current operation. The current version [1] is already heavily pointer-based, but still contains boilerplate text in Section 3 ("Current Dispatch Packet"). For example, the "Canon load order" list is hardcoded:

1. PoT.md
2. SoP.md

   ...

Under a strict pointer-first constitution, TASK.md should not list these files. It should reference ops/lib/manifests/CONTEXT.md as the authority on what must be loaded. However, TASK.md is also a human-readable contract. A balance must be struck: the DP should list the files for *verification* (Did I load what was required?) but defer to the manifest for *definition* (What is required?).

4.3 Application to Agents

The agent definitions (R-AGENT-01 to R-AGENT-06) are currently defined in markdown files in docs/library/agents/.[1] They contain sections like "Scope Boundary" and "Pointers."

- **Current:** R-AGENT-01 lists "Constitution: PoT.md".
- **Pointer-First Refinement:** The agent should not inherently know *how* to interpret PoT.md; it should know that PoT.md is the constraint. The interpretation logic (e.g., "Stop work on violation") is part of the PoT.md content, not the agent's distinct personality.

The pointer-first model reduces the "token weight" of agent definitions, making them lightweight pointers that dynamically inherit the system's current governance stance.

---

5. Strategic Unification: The "One Truth" Architecture

The report "Unifying LLM Context and Operations" [1] proposes a specific architectural consolidation to resolve the "Two Truths" problem. This strategy relies on a hierarchical inheritance model for manifests.

5.1 The Hierarchical Manifest Model

Instead of maintaining CONTEXT.md and LLMS.md as separate, potentially conflicting lists, the system will adopt a layered approach:

Layer	Manifest	Content	Purpose
Layer 0	**CORE**	PoT.md, SoP.md, TASK.md, docs/MAP.md	The "Physical Laws." Immutable constraints required for *any* operation.
Layer 1	**OPS**	Inherits **CORE** + ops/bin/*, tools/*, docs/MANUAL.md	Operational mechanics. Required for maintenance and platform work.
Layer 2	**DISCOVERY**	Inherits **OPS** + docs/library/* (Agents, Tasks, Skills)	Full system comprehension. Used for planning and architectural

			review.

5.2 The Unified Logic Engine: synthesize.sh

The proposal introduces a conceptual or literal unification of the binary logic. Currently, ops/bin/context and ops/bin/llms are separate scripts with separate logic. The unification proposes a centralized logic engine (potentially named synthesize.sh or integrated into an upgraded ops/bin/context) that:

1. Resolves the hierarchical manifests (e.g., "Give me Layer 1").
2. Enforces hazards centrally (e.g., "If Layer < 2, strictly forbid docs/library/").
3. Standardizes output formatting (ensuring llms.txt and dump outputs share the same structure).

5.3 Deprecation of Specialized Bundles

The proliferation of bundles like llms-governance.txt and llms-ops.txt is identified as a source of drift. The unification strategy mandates their deprecation. Instead of static text files checked into the repo, these contexts should be generated on-demand or reduced to a minimal set of standard artifacts:

- llms.txt: The entry point (Pointer).
- llms-core.txt: The minimal execution context (Layer 0).
- llms-full.txt: The complete discovery context (Layer 2).

This reduction simplifies the "Allowlist Rule" for DPs. Instead of allowlisting five different text files, the DP only needs to allowlist the entry point and the specific layer required for the task.

---

6. Implementation Mechanics: DP-OPS-0059

Dispatch Packet DP-OPS-0059 is the vehicle for executing this architectural shift. Based on the constraints provided in the user query and the insights from the research material, we can reconstruct the necessary components of this DP.

6.1 Objective and Logic

The objective is "Unifying LLM Context and Operations." The logic must adhere to PoT.md. This means the DP itself must be:

- **Pointer-First:** It cannot contain long preamble text explaining the unification; it must point to [1] or the relevant specs.
- **Anti-Drift:** It must ensure that the new llms.txt and context binaries do not deviate from the File System SSOT.

- **Equilibrium Maintenance:** The task is not done until SoP.md is updated and the system is in a stable state.

6.2 The llms.txt Standard Integration

The Stela platform adopts the llms.txt standard [2] to facilitate this. The standard dictates a specific format:

- **H1 Header:** Project Name.
- **Blockquote:** Summary.
- **File Lists:** H2 headers with links to markdown files.

In Stela's implementation, llms.txt becomes the "Router" for the pointer-first constitution. It does not contain the context; it points to the bundles (llms-core.txt, llms-full.txt) that contain the context.

Crucial Implementation Detail: The ops/bin/llms binary must be updated to generate this file dynamically based on the repository state (HEAD hash, active branch).[1] This ensures that the "Discovery Truth" is never stale.

6.3 The Allowlist Rule

The user query specifies a strict addendum:

> "If the DP plan includes an llms invocation... the DP allowlist must include: llms.txt, llms-small.txt, llms-full.txt, llms-ops.txt, llms-governance.txt."

This constraint highlights the current fragility of the system. These files must be explicitly allowed because the tools/lint/dp.sh linter is likely configured to reject broad file access. By explicitly listing them, the Operator overrides the linter's caution.

- **Insight:** DP-OPS-0059 should ideally *remove* the need for this manual list in future DPs by standardizing the output into a single predictable directory or archive, or by updating the linter to recognize the unified bundle structure as safe. However, *for the execution of DP-OPS-0059 itself*, the specific list must be adhered to as per the "Absolute Literalism" standard.[1]

6.4 Canon Surface Replacement

The query mandates: "If the DP replaces a canon surface... the DP must inline the full authoritative replacement text OR point to an in-repo file path."

Since DP-OPS-0059 is redefining how context is loaded, it will likely modify ops/lib/manifests/CONTEXT.md and TASK.md.

- **Modification to TASK.md:** The "Required Context Load" section of TASK.md [1] currently lists specific files. DP-OPS-0059 implies changing this to a pointer to the new unified

manifest system.

- **Modification to PoT.md:** The "Entry Points" section of PoT.md lists llms.txt.[1] This remains valid, but the definition of what llms.txt *is* (a generated artifact vs. a static file) might need clarification in docs/ops/specs/binaries/llms.md.

---

7. Operational Governance and Policing

The unification of context requires a corresponding update to the policing mechanisms: the linters.

7.1 tools/lint/context.sh

Currently, this linter verifies that files in ops/lib/manifests/CONTEXT.md exist and checks for "Semantic Contamination" (e.g., referencing docs/library/ in the context).[1] With the unification, context.sh must become aware of the layers.

- If verifying Layer 0 (Core), it must strictly reject library references.
- If verifying Layer 2 (Discovery), it must *allow* library references but verify their pointer integrity (No "Dead Ends" [1]).

7.2 tools/lint/dp.sh

This linter enforces the structure of the Dispatch Packet. It ensures the "Freshness Gate" and "Receipt" blocks are present.[1] DP-OPS-0059 introduces a challenge: if the DP changes the structure of TASK.md (the template for DPs), the linter might fail when validating the new format.

- **Insight:** The unification process must be atomic. The DP must update the linter (tools/lint/dp.sh) *in the same transaction* that it updates the TASK.md template, ensuring the policing logic matches the new constitutional standard.

7.3 The "Anti-Drift" Directive

Stela's agents are programmed to "Actively monitor for structural anomalies".[1] A unified context engine significantly strengthens this capability. When an agent can see the "Full Truth" (Layer 2) via llms-full.txt, it can detect if a file in ops/ deviates from its description in docs/ops/specs/. Currently, an agent restricted to the Execution Truth (Layer 0) cannot perform this check because it cannot see the specifications in docs/. Unification empowers the "Integrator" agent [1] to perform deep structural audits that were previously impossible without manual human intervention.

---

8. Implementation Artifact Analysis: DP-OPS-0059

Based on the synthesis of the user query and the research, the Dispatch Packet DP-OPS-0059 serves as the implementation blueprint. The following analysis breaks down its required components according to the "Absolute Literalism" and "Linguistic Precision" standards of Stela.

8.1 Freshness and Preflight

The DP must anchor itself to the current main branch state.

- **Base HEAD:** 2dafc32f (derived from [1]).
- **Work Branch:** work/dp-ops-0059-unify-context.
- **Preflight:** Must run tools/lint/dp.sh to ensure the DP itself is valid before execution starts.

8.2 Scope and Safety

- **Objective:** "Unify LLM Context and Operations."
- **Allowlist:** As mandated, must include llms.txt, llms-small.txt, llms-full.txt, llms-ops.txt, llms-governance.txt because the plan involves invoking llms (the binary that generates these files).
- **Invariants:** The "Context Hazard" rule remains in effect for the *execution* phase. The unification adds the capability to safely reference the hazardous material for *discovery*, but does not permit it in the *execution* manifest (ops/lib/manifests/CONTEXT.md).

8.3 Execution Plan Construction

The execution plan must proceed in a logical order to prevent the "Two Truths" conflict from breaking the build during the transition.

1. **Refactor Manifests:** Create the Layered Manifest structure (Core, Ops, Discovery) in ops/lib/manifests/.
2. **Update Binaries:** Update ops/bin/llms to generate the new llms.txt structure and bundle files based on the new manifests.
3. **Update Linters:** Align tools/lint/context.sh and tools/lint/llms.sh to validate the new hierarchical structure.
4. **Update Canon:** Update TASK.md to reference the new manifests instead of hardcoded lists.
5. **Receipt:** Run the full suite of verification tools (verify.sh, lint/truth.sh, etc.) and capture the dump proof.

8.4 Receipt and Closeout

The DP requires a specific "Receipt" block.[1] This is not just a log; it is a cryptographic proof of work. It requires git diff --stat to prove that only the allowlisted files were touched.

- **Critical Constraint:** The "Zero-byte check" (test -s <dump_payload_path>) ensures that the state capture (the "Truth" passed to the agent) is not empty. In a pointer-first architecture, an empty dump is a catastrophic failure because there is no duplicated text to fall back on.

---

9. Future Implications of a Flat Truth Architecture

The successful execution of DP-OPS-0059 transitions Stela from a fragmented epistemic model to a "Flat Truth" architecture.

9.1 The End of Ghost Canon

By enforcing pointer-first references in TASK.md and Agent definitions, Stela eliminates the synchronization cost of governance updates. Changing PoT.md immediately propagates to every agent and every new task, as they all "dereference" the same file at runtime.

9.2 The "Context is a File" Philosophy

The doctrine "Context is not memory; it is a file" [1] is reinforced. Unification ensures that this file is deterministically generated and cryptographically verifiable (via git hash). This transforms the AI from a "black box" thinker into a verifiable state machine.

9.3 Improved Agent Agency

With the "Discovery Truth" (Layer 2) properly integrated and accessible via llms.txt, agents like the "Architect" [1] gain legitimate, safe access to the platform's full capabilities without violating safety protocols. This enables higher-order reasoning, self-correction, and autonomous "Anti-Drift" monitoring, fulfilling the platform's ultimate design goals.

Conclusion

The "Pointer-first agent constitution" is not merely a stylistic preference; it is a structural necessity for maintaining epistemic integrity in a repository-resident AI system. By unifying the execution and discovery pipelines through DP-OPS-0059, Stela eliminates the "Two Truths" hazard, eradicates "Ghost Canon," and establishes a robust, scalable foundation for autonomous operation. The rigorous application of the Policy of Truth—mediated by strict linters and hierarchical manifests—ensures that as the system expands, its governance remains coherent, verifiable, and absolute.

Appendix: Implementation Artifact (DP-OPS-0059)

The following Dispatch Packet constitutes the executable logic derived from the analysis above, formatted strictly according to the TASK.md template and user query constraints.

DP-OPS-0059: Unifying LLM Context and Operations

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/dp-ops-0059-unify-context

Base HEAD: 2dafc32f

Required local re-check (worker runs; paste outputs in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse --short HEAD
- git status --porcelain

Preconditions:

- No commits on main.
- Working tree must be clean before execution begins.
- If Base HEAD changes, regenerate session artifacts from canonical tools before proceeding.
  STOP conditions:
- STOP if any mismatch (branch, Base HEAD, missing work branch).
- STOP if working tree is dirty before execution begins.
- STOP if told to create or switch branches.

3.1.1 DP Preflight Gate (Run Before Any Edits)

Purpose:

- Catch malformed DP or TASK schema before work begins.
  Worker runs (paste outcome in RESULTS):
- bash tools/lint/dp.sh --test
- bash tools/lint/dp.sh TASK.md

- bash tools/lint/task.sh

STOP if any preflight check fails.

3.2 Required Context Load (Read Before Doing Anything)

3.2.1 Canon load order (always)

Worker must confirm loaded before edits begin:

1. PoT.md
2. SoP.md
3. TASK.md
4. docs/MAP.md
5. docs/MANUAL.md
6. ops/lib/manifests/CONTEXT.md

Notes:

- Worker does not read OPEN.
  OPEN is for Integrator state refresh and for receipts.
- Disposable artifacts must not be referenced or included.
- Worker input is the DP text only.
- DP writer must not attach or cite disposable artifacts.
- DP writer must not embed pasted bundles.

3.2.2 DP-scoped load order (per DP)

- tools/lint/dp.sh
- tools/lint/task.sh
- ops/bin/prune
- docs/ops/specs/surfaces/task.md
- docs/ops/specs/binaries/prune.md
- docs/ops/specs/binaries/open.md (pointer reference only; OPEN behavior is not modified in this DP)

3.3 Scope and Safety

Objective: Unify LLM context and operations by establishing a pointer-first constitution, consolidating manifest logic into hierarchical layers (Core, Ops, Discovery), and eliminating ghost canon in TASK.md.

In scope:

- Refactor ops/lib/manifests/ to support hierarchical layers.

- Update ops/bin/llms to generate llms-core.txt and llms-full.txt from the new manifests.
- Update TASK.md to use pointer-first context loading (removing hardcoded file lists).
- Update tools/lint/context.sh and tools/lint/llms.sh to enforce the new structure.
  Out of scope:
- Changes to ops/bin/open logic (other than manifest consumption).
- Changes to Agent definitions (R-AGENT-*) unless strictly required by manifest changes.
  Safety and invariants:
- Context Hazard: Library files must remain excluded from the Execution Context (Layer 0/1).
- Flat Truth: llms.txt must remain at the repo root.
- Anti-Drift: All generated bundles must match the manifests exactly.

Target Files allowlist (hard gate):

- ops/lib/manifests/CONTEXT.md
- ops/lib/manifests/LLMS.md
- ops/bin/llms
- tools/lint/context.sh
- tools/lint/llms.sh
- TASK.md
- llms.txt
- llms-small.txt
- llms-full.txt
- llms-ops.txt
- llms-governance.txt

3.4 Execution Plan (A–E)

3.4.1 State

Populate during execution; do not pre-fill in TASK.md.

3.4.2 Request

Populate during execution; do not pre-fill in TASK.md.

3.4.3 Changelog

Populate during execution; do not pre-fill in TASK.md.

3.4.4 Patch / Diff

Populate during execution; do not pre-fill in TASK.md.

3.4.5 Receipt (Proofs to collect) — MUST RUN

-./ops/bin/open --out=auto --dp="DP-OPS-0059"

-./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle

- Zero-byte check: test -s <dump_payload_path>
- bash tools/lint/context.sh
- bash tools/lint/style.sh
- bash tools/lint/truth.sh
- bash tools/lint/dp.sh --test
- bash tools/lint/dp.sh TASK.md
- bash tools/lint/task.sh
- bash tools/lint/llms.sh
  -./tools/verify.sh
- git diff --name-only
- git diff --stat
- Required pasted outputs: receipts, verification outcomes, and diff output.
- Mandatory Closing Block required in RESULTS.

4. Closeout

- Execute docs/MANUAL.md Closeout Cycle in order (Verify, Harvest, Refresh, Log, Prune).
- Update SoP.md with a DP entry at completion, including objective summary and verification commands run.
- Protocol order for closeout: Verify -> Generate Results -> COMMIT (Operator Only) -> Prune.
- Run prune hygiene at the end of the closeout sequence:./ops/bin/prune --dp=DP-OPS-0059 --scrub.
- Use./ops/bin/prune --reset-task only for explicit TASK baseline reset after Work Log clear.
- Ensure the next session begins with refreshed session artifacts and matching receipts.

Works cited

1. Unifying LLM Context and Operations.md
2. What is llms.txt? Why it's important and how to create it for your docs – GitBook Blog, accessed February 12, 2026, https://www.gitbook.com/blog/what-is-llms-txt
3. llms-txt: The /llms.txt file, accessed February 12, 2026, https://llmstxt.org/